Q7: Background Tasks & Async Architecture

> ***Question:*** *How does this backend handle long-running or computationally heavy tasks? Are there background task workers (like Celery), message queues (like RabbitMQ), or native async background tasks? Explain the flow of a task that is deferred to the background rather than processed synchronously in the main request loop.*

Short Answer

There is no Celery, no RabbitMQ, and no separate worker process. All deferred work runs inside the **same Python process** as the FastAPI server, using three distinct mechanisms:

Mechanism	Used For	Runs On
APScheduler (`AsyncIOScheduler`)	Timed/recurring jobs (schedule generation, rotation cycling, midnight reset)	FastAPI's own asyncio event loop
`asyncio.create_task()`	Long-lived listener loop (Redis Pub/Sub → WebSocket relay)	FastAPI's own asyncio event loop
Inline async logic inside request handlers	Heavy multi-step operations triggered by hardware events (auto-dispatch)	Awaited synchronously within the HTTP request

FastAPI's native `BackgroundTasks` is **not used** in this codebase — the patterns above are preferred because they give finer control over database sessions and error handling.

Mechanism 1: APScheduler — Scheduled Jobs

File: `app/services/scheduler.py`

How it starts

During app startup the `lifespan` context manager in `app/main.py` calls `start_scheduler()` :

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        start_scheduler()          # registers cron + interval jobs, starts the scheduler
    except Exception as e:
        logger.warning(f"Scheduler failed to start (non-critical): {e}")
    ...
    yield
    scheduler.shutdown(wait=False) # clean shutdown on exit
```

`start_scheduler()` creates an `AsyncIOScheduler` instance — this is the asyncio-native variant of APScheduler. It plugs into the **already-running event loop** created by uvicorn, so scheduled coroutines are awaited on the same loop that serves HTTP requests. No new threads, no new processes.

The three registered jobs

```
scheduler.add_job(
    scheduled_assignment_job,          # coroutine
    CronTrigger(hour=5, minute=30),    # runs at 05:30 every day
    id="daily_assignments",
    replace_existing=True
)

scheduler.add_job(
    rotation_manager_job,
    'interval',
    minutes=5,                         # runs every 5 minutes
    id="rotation_processor",
    replace_existing=True,
    misfire_grace_time=300,            # allow up to 5-min late fire
    max_instances=1,                   # never run two copies simultaneously
)

scheduler.add_job(
    midnight_reset_job,
    CronTrigger(hour=0, minute=0),     # runs at midnight
    id="midnight_reset",
    replace_existing=True
)
```

Job 1: `scheduled_assignment_job` — Daily Schedule Generation (05:30)

The heaviest job. It calls `generate_daily_schedule(db, date.today())` which:

1. Opens a fresh `AsyncSession` (not borrowed from any request — dedicated session)
2. Queries all active `Route` rows
3. Queries all `OFF_DUTY` drivers and `FREE` vehicles
4. For each `(route × shift)` pair that has ≥ 3 drivers and ≥ 2 vehicles available:
   - Creates 3 `RotationAssignment` rows (DRIVER_1 / DRIVER_2 / DRIVER_3)
   - Creates up to 6 `Trip` rows (2 per driver: OUTBOUND + INBOUND)
   - Calls `await db.flush()` after each assignment to get the auto-increment `id` before linking trips
5. Calls `await db.commit()` once at the end

The guard at the top prevents double-generation if today's assignments already exist:

```
existing_count = await db.scalar(
    select(func.count()).select_from(RotationAssignment).where(
        RotationAssignment.shift_date == target_date
    )
)
if existing_count and not regenerate:
    return []   # already done, skip
```

Why this is safe: The scheduler fires on the event loop, but `await` yields control back to the event loop between DB round-trips. HTTP requests continue to be served in those yield windows. The job is not blocking.

Job 2: `rotation_manager_job` — Rotation Cycling (every 5 min)

Calls `process_rotations(db)` . This implements the ping-pong driver swap algorithm:

1. Fetches all `RotationAssignment` rows for the current shift window
2. Groups them by route → gets the D1/D2/D3 assignment for each route
3. Checks how many hours have elapsed since shift start:

```
hours_since_start in [1, 3)  → if D1 is still active → swap D1 ↔ D3
hours_since_start in [3, 5)  → if D2 is still active → swap D2 ↔ D1
hours_since_start in [5, 7)  → if D3 is still active → swap D3 ↔ D2
```

4. `_perform_swap(outgoing, incoming)` flips `is_active` flags, updates `Driver.status` (ON_BREAK / ACTIVE), and inserts a `DriverExchange` audit row
5. One `await db.commit()` at the end covers all swaps in the same run

`max_instances=1` on this job prevents a second firing from overlapping with a still-running swap — important because the swap commits database state that the next run depends on.

Job 3: `midnight_reset_job` — Driver Counter Reset (00:00)

Simple bulk update. Opens a session, loads every `Driver` , zeroes out five counters, commits:

```
d.total_trips_today = 0
d.break_time_remaining = settings.BREAK_TIME_PER_SHIFT
d.total_break_time_today = 0.0
d.trips_since_last_break = 0
d.current_break_number = 0
```

No business logic — purely a daily state reset so fatigue scores and break eligibility start fresh.

Mechanism 2: `asyncio.create_task()` — Redis Pub/Sub Listener

File: `app/core/sockets.py`

During startup, `ConnectionManager.init_redis()` is called:

```
async def init_redis(self):
    self.redis = Redis.from_url(settings.REDIS_URL, decode_responses=True)
    self.pubsub = self.redis.pubsub()
    await self.pubsub.subscribe("broadcast", "alerts")
    self._redis_listener_task = asyncio.create_task(
        self._run_redis_listener_supervisor()
    )
```

`asyncio.create_task()` schedules the supervisor coroutine on the event loop **without blocking** startup. The task runs indefinitely in the background — one concurrent task alongside all HTTP request coroutines.

The supervisor loop

```
async def _run_redis_listener_supervisor(self):
    while True:
        try:
            await self._redis_listener()       # blocks here until Redis drops
        except asyncio.CancelledError:
            raise                              # clean shutdown
        except Exception as e:
            logger.error(f"Supervisor: {e}")

        await asyncio.sleep(5)                 # wait before reconnect attempt
        await self._ensure_redis_connected()   # recreate Redis connection
        self.pubsub = self.redis.pubsub()
        await self.pubsub.subscribe("broadcast", "alerts")
```

The listener itself is an `async for` loop over the Redis Pub/Sub message stream:

```
async def _redis_listener(self):
    async for message in self.pubsub.listen():
        if message["type"] == "message":
            data = json.loads(message["data"])
            target_role = data.get("role")
            if target_role:
                await self.broadcast_to_role(target_role, data)
            else:
                await self.broadcast_to_all(data)
```

When any part of the application publishes to the `"alerts"` Redis channel (e.g., a hardware endpoint detects an over-speed GPS event), the listener picks it up and calls `broadcast_to_role()` — which iterates the in-memory set of connected WebSocket clients for that role and pushes the JSON payload to each.

Why `asyncio.create_task()` here instead of APScheduler: This is a permanently-running listener, not a periodic job. `create_task` is the idiomatic asyncio primitive for "start this coroutine and let it run forever."

Mechanism 3: Inline Heavy Work Within Request Handlers

Some operations are computationally significant but are still performed **synchronously within the HTTP request** — they are awaited before the response is returned. The most notable example is `trigger_auto_dispatch()` , called from `POST /api/v1/hardware/camera` .

This is **not** a background task — the HTTP client waits for it. But it is structured like a service call to keep the endpoint handler clean.

Why it stays in the request

- The client (an ESP32-CAM) needs to know whether the dispatch succeeded to log the outcome on-device
- The response `{"message": "Crowding updated", "score": 0.92}` is only returned after all DB writes are committed

• The operation is fast enough (< 100 ms even with FOR UPDATE locks) that deferring it would add unnecessary complexity

Flow of `trigger_auto_dispatch(trip_id, db)`

```
POST /hardware/camera arrives
        ■
        ■■■ [sync] fetch Trip
        ■■■ [sync] fetch Vehicle
        ■■■ [sync] calculate crowding_score
        ■■■ [sync] db.add(CameraReading)
        ■
        ■■■ crowding_score >= 0.90 ?
              ■
              YES ■■■ trigger_auto_dispatch(trip_id, db)
                            ■
                            ■■■ SELECT Driver ... FOR UPDATE (locks row)
                            ■■■ SELECT Vehicle ... FOR UPDATE (locks row)
                            ■■■ SELECT Route (for duration)
                            ■■■ db.add(new_trip)
                            ■■■ db.flush()          ← SQL sent, no commit yet
                            ■                         new_trip.id now available
                            ■■■ db.add(DriverExchange)
                            ■■■ db.commit()         ← 1st commit (dispatch data)
              ■
              ■■■ db.add(CrowdingEvent)
              ■■■ db.commit()                       ← 2nd commit (sensor data)
              ■
              ■■■ WebSocket broadcast to MANAGER
              ■
              ■■■ return HTTP 200
```

The key insight: both the auto-dispatch commit and the sensor-data commit happen **within the same HTTP request lifecycle**, sharing the same `AsyncSession` (`db`). The session is injected by `get_db()` and cleaned up in its `finally` block after the response is sent.

What Is NOT Used

Technology	Status	Reason
Celery	Not used	No need for distributed task queues; all jobs fit on one process
RabbitMQ	Not used	Redis Pub/Sub is sufficient for the WebSocket fan-out use case
FastAPI `BackgroundTasks`	Not used	Would require passing a new DB session into the background function, and provides no return value — APScheduler + inline async gives cleaner control

Technology	Status	Reason
`asyncio.gather()`	Not used in handlers	All DB operations are sequential within each request; no parallel DB calls
Separate worker processes	Not used	Single-server deployment; horizontal scaling not yet required

---

Session Management in Background Jobs

Every background job that touches the database opens its **own** `AsyncSession` via `AsyncSessionLocal()` — it does not reuse any request-scoped session:

```
async def scheduled_assignment_job():
    async with AsyncSessionLocal() as db:
        try:
            await generate_daily_schedule(db, date.today())
        except Exception as e:
            logger.error(f"Failed: {e}")
            # AsyncSessionLocal context manager rolls back on exception
```

The `async with` block ensures the session is closed (and the connection returned to the pool) whether the job succeeds or raises. This is identical to what `get_db()` does for HTTP requests.

Failure Modes and Recovery

Job	What Happens on Failure	Recovery
`scheduled_assignment_job`	Exception logged; no assignments generated	Admin can trigger `POST /admin/rotations/generate` manually; idempotency guard prevents duplicates
`rotation_manager_job`	Exception logged; no swap this cycle	Retried 5 minutes later; `max_instances=1` ensures no overlap
`midnight_reset_job`	Exception logged; counters not reset	No automatic recovery; counters remain from previous day until next midnight
Redis listener	Supervisor catches exception; waits 5 s; reconnects	Automatic — indefinite reconnect loop; WS push is disabled only during the gap
`trigger_auto_dispatch`	Returns `False` ; endpoint still returns 200	Caller logs warning; camera reading and crowding event are still persisted

Generated: 2026-04-16